

Profissionais de Dados no Brasil: O que usam vs. o que buscam

Análise comparativa — State of Data Brazil 2021–2024 × Google Trends

Grupo — [preencher nomes]

22/06/2026

Índice

1	Introdução e Pergunta de Pesquisa	2
2	Bases de Dados	2
2.1	Base 1 — State of Data Brazil 2021–2024	2
2.2	Base 2 — Google Trends via API pública (pytrends)	3
3	Limpeza, Padronização e Integração	4
4	Modelo de Recuperação da Informação — TF-IDF	5
4.1	Justificativa da escolha	5
4.2	Construção do índice	5
4.3	Sistema de busca de perfis	6
4.3.1	Exemplo 1 — Perfil Python + Cientista de Dados em SP	6
4.3.2	Exemplo 2 — Analista de Dados SQL em estados do Nordeste	7
4.3.3	Exemplo 3 — Engenheiro de Dados com experiência avançada	7
4.4	Perfis de estados via TF-IDF	7
5	Resultados e Visualizações	8
5.1	Evolução das linguagens no State of Data (2021–2024)	8
5.2	Google Trends: interesse em linguagens de dados no Brasil	9
5.3	Comparativo: adoção real vs. interesse no Google (por ano)	10
5.4	Distribuição salarial por linguagem principal	12
5.5	Correlação: adoção de Python vs. interesse no Google por estado	13
5.6	Top 10 estados por número de profissionais de dados	14
6	Discussão Crítica	15
7	Conclusão	16
8	Referências	16

1 Introdução e Pergunta de Pesquisa

O mercado de dados brasileiro cresceu de forma acelerada entre 2021 e 2024. A pesquisa **State of Data Brazil** captura, ano a ano, o perfil de quem trabalha com dados no país — suas ferramentas, salários, cargos e distribuição regional. Mas será que o **interesse público** por essas tecnologias, medido pelo Google, acompanha o que os profissionais realmente usam?

Este trabalho responde às seguintes perguntas:

1. **Linguagens e ferramentas:** As tecnologias mais usadas pelos profissionais (State of Data) são também as mais buscadas no Google por estado e ao longo do tempo?
 2. **Tendência temporal:** O crescimento do uso de Python e SQL entre profissionais (2021–2024) é paralelo ao aumento de buscas no Google?
 3. **Distribuição regional:** Há estados onde o interesse online supera a presença de profissionais — potenciais mercados emergentes?
 4. **Recuperação de perfis (TF-IDF):** Dado um perfil de competências desejado, quais profissionais do State of Data são os mais relevantes?
-

2 Bases de Dados

2.1 Base 1 — State of Data Brazil 2021–2024

Pesquisa anual conduzida pela comunidade Data Hackers com profissionais de dados no Brasil. Consolidamos as quatro edições (2021, 2022, 2023, 2024) em um único dataset `.parquet` via `src/coleta/consolida_state_of_data.py`.

```
from pathlib import Path
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

plt.rcParams.update({
    "figure.dpi": 120,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "font.family": "sans-serif",
})

PROC = Path("data/processed")
RAW = Path("data/raw")
```

```
df_sod = pd.read_parquet(PROC / "sod_limpo.parquet", engine="pyarrow")

print(f"State of Data: {df_sod.shape[0]:,} respondentes × {df_sod.shape[1]} colunas")
print(f"Anos: {sorted(df_sod['ano'].unique())}")
print(f"Estados (UF): {df_sod['uf'].nunique()}")
df_sod[["ano", "uf", "cargo_grupo", "linguagem_mais_usada",
        "faixa_salarial", "salario_medio", "nivel_ensino"]].head(5)
```

State of Data: 16,962 respondentes × 13 colunas
 Anos: [np.int64(2021), np.int64(2022), np.int64(2023), np.int64(2024)]
 Estados (UF): 27

	ano	uf	cargo_grupo	linguagem_mais_usada	faixa_salarial	salario_medio	
0	2021	CE	Outro	NaN	de R\$ 4.001/mês a R\$ 6.000/mês	5000.5	1
1	2021	BA	Outro	NaN	de R\$ 6.001/mês a R\$ 8.000/mês	7000.5	1
2	2021	SC	Outro	NaN	de R\$ 8.001/mês a R\$ 12.000/mês	10000.5	1
3	2021	SP	Outro	NaN	de R\$ 12.001/mês a R\$ 16.000/mês	14000.5	1
4	2021	SC	Outro	NaN	de R\$ 8.001/mês a R\$ 12.000/mês	10000.5	1

2.2 Base 2 — Google Trends via API pública (pytrends)

Coletamos o índice de interesse de busca (0–100) para 15 termos relacionados a dados e tecnologia no Brasil, de 2021 a 2024, com recorte por estado brasileiro. A coleta foi feita via `src/coleta/coleta_externa.py` usando a biblioteca `pytrends`, sem autenticação.

Termos coletados:

- **Linguagens:** Python, SQL, R, Scala, Julia
- **Ferramentas:** Power BI, Tableau, Apache Spark, dbt, Databricks
- **Carreiras:** cientista de dados, engenheiro de dados, analista de dados, machine learning, inteligência artificial

```
df_serie = pd.read_csv(RAW / "trends_serie_temporal.csv", parse_dates=["date"])
df_estado = pd.read_parquet(PROC / "trends_limpo.parquet", engine="pyarrow")
df_integ = pd.read_parquet(PROC / "base_integrada.parquet", engine="pyarrow")

print(f"Série temporal: {len(df_serie):,} registros | {df_serie['termo'].nunique()} termos")
print(f"Por estado: {len(df_estado):,} registros | {df_estado['uf'].nunique()} UFs")
print(f"Base integrada: {len(df_integ):,} pares UF×ano")
df_serie.head(5)
```

Série temporal: 3,150 registros | 15 termos
 Por estado: 1,367 registros | 27 UFs
 Base integrada: 95 pares UF×ano

	date	termo	interesse	grupo	ano
0	2020-12-27	Python	11	linguagens	2020
1	2021-01-03	Python	11	linguagens	2021
2	2021-01-10	Python	12	linguagens	2021
3	2021-01-17	Python	12	linguagens	2021
4	2021-01-24	Python	13	linguagens	2021

3 Limpeza, Padronização e Integração

O script `src/limpeza/integra.py` executou as seguintes transformações:

State of Data

- Extração da sigla UF a partir do formato "São Paulo (SP)" usando expressão regular
- Conversão das faixas salariais textuais para o ponto médio numérico (ex: "R\$4.001 a R\$6.000" → 5000.5)
- Agrupamento de cargos em categorias padronizadas (Analista de Dados, Cientista de Dados, Engenheiro de Dados, etc.)

Google Trends

- Normalização dos nomes de estado (trata variantes como "State of São Paulo")
- Remoção de estados sem dados suficientes (interesse = 0)

Integração

Agregação do State of Data por UF × ano e união (inner join) com o interesse médio do Google Trends por grupo de termos, resultando em **95 pares UF × ano** com informações de ambas as fontes.

```
df_integ.sort_values(["uf", "ano"]).head(10)
```

	uf	ano	n_respondentes	salario_mediano	linguagem_top	cargo_top	pct_python	pct_sql	t
0	AC	2023	3	14000.5	SQL	Outro	0.000000	100.000000	2
1	AC	2024	1	27500.5	NaN	Outro	0.000000	0.000000	2
2	AL	2021	14	7000.5	Python	Outro	62.500000	37.500000	2
3	AL	2022	32	7000.5	Python	Outro	57.142857	33.333333	2
4	AL	2023	31	7000.5	Python	Outro	47.619048	33.333333	2
5	AL	2024	23	5000.5	Python	Outro	50.000000	35.714286	2
6	AM	2022	26	5000.5	Python	Outro	63.157895	10.526316	2
7	AM	2023	29	7000.5	Python	Outro	71.428571	19.047619	2
8	AM	2024	23	10000.5	Python	Outro	52.941176	41.176471	2
9	AP	2023	5	7000.5	Python	Outro	100.000000	0.000000	3

4 Modelo de Recuperação da Informação — TF-IDF

4.1 Justificativa da escolha

Utilizamos o modelo **Vetorial com TF-IDF** (Term Frequency–Inverse Document Frequency) com similaridade do cosseno. A escolha se justifica por:

- Adequação ao problema: os perfis dos respondentes são descrições textuais de atributos (língua, cargo, estado, experiência), e o TF-IDF captura quais termos distinguem um perfil dos demais
- Implementação eficiente via `scikit-learn`, amplamente usada em produção
- Interpretabilidade dos resultados: os pesos TF-IDF são diretamente legíveis

4.2 Construção do índice

Cada respondente do State of Data é representado como um **documento de texto** que concatena seus atributos: língua principal, cargo, UF, nível de ensino e tempo de experiência.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

def perfil_para_texto(row: pd.Series) -> str:
    """Converte atributos de um respondente em texto para o TF-IDF."""
    campos = [
        str(row.get("língua_mais_usada", "") or ""),
        str(row.get("cargo_grupo", "") or ""),
        str(row.get("uf", "") or ""),
        str(row.get("nível_ensino", "") or ""),
        str(row.get("tempo_experiência_dados", "") or ""),
    ]
    return " ".join(c for c in campos if c and c != "nan").lower()

# Cria o corpus
corpus = df_sod.apply(perfil_para_texto, axis=1)
corpus_valido = corpus[corpus.str.strip() != ""]
idx_validos = corpus_valido.index

# Treina o TF-IDF
vectorizer = TfidfVectorizer(
    ngram_range=(1, 2),    # unigramas e bigramas
    min_df=3,              # ignora termos raros
    max_df=0.95,           # ignora termos ubíquos
)
tfidf_matrix = vectorizer.fit_transform(corpus_valido)

print(f"Corpus: {tfidf_matrix.shape[0]:,} documentos × {tfidf_matrix.shape[1]:,} termos")
print(f"\nExemplos de documentos:")
```

```
for doc in corpus_valido.iloc[:3]:
    print(f"  '{doc}')
```

Corpus: 16,962 documentos × 419 termos

Exemplos de documentos:

```
'outro ce pós-graduação mais de 10 anos'
'outro ba pós-graduação de 2 a 3 anos'
'outro sc pós-graduação de 1 a 2 anos'
```

4.3 Sistema de busca de perfis

```
def buscar_perfis(query: str, n: int = 5) -> pd.DataFrame:
    """
    Dado uma query em texto, retorna os n perfis mais similares
    no State of Data, ordenados por similaridade do cosseno.
    """
    q_vec = vectorizer.transform([query.lower()])
    scores = cosine_similarity(q_vec, tfidf_matrix).flatten()

    top_idx = scores.argsort()[::-1][:n]
    idx_originais = idx_validos[top_idx]

    resultado = df_sod.loc[idx_originais, [
        "ano", "uf", "cargo_grupo", "linguagem_mais_usada",
        "nivel_ensino", "tempo_experiencia_dados", "faixa_salarial"
    ]].copy()
    resultado.insert(0, "similaridade", scores[top_idx].round(3))
    return resultado.reset_index(drop=True)
```

4.3.1 Exemplo 1 — Perfil Python + Cientista de Dados em SP

```
buscar_perfis("python cientista de dados SP superior completo")
```

	similaridade	ano	uf	cargo_grupo	linguagem_mais_usada	nivel_ensino	tempo_experiencia
0	0.808	2024	SP	Cientista de Dados	Python	Pós-graduação	de 5 a 6 anos
1	0.808	2022	SP	Cientista de Dados	Python	Pós-graduação	de 1 a 2 anos
2	0.808	2023	SP	Cientista de Dados	Python	Pós-graduação	de 3 a 4 anos
3	0.808	2023	SP	Cientista de Dados	Python	Pós-graduação	de 3 a 4 anos
4	0.808	2024	SP	Cientista de Dados	Python	Pós-graduação	de 3 a 4 anos

4.3.2 Exemplo 2 — Analista de Dados SQL em estados do Nordeste

```
buscar_perfis("sql analista de dados BA CE PE")
```

	similaridade	ano	uf	cargo_grupo	linguagem_mais_usada	nivel_ensino	tempo_experien
0	0.649	2023	BA	Analista de Dados	SQL	Graduação/Bacharelado	de 1 a 2
1	0.649	2023	BA	Analista de Dados	SQL	Graduação/Bacharelado	de 1 a 2
2	0.649	2024	BA	Analista de Dados	SQL	Graduação/Bacharelado	de 5 a 6
3	0.649	2023	BA	Analista de Dados	SQL	Graduação/Bacharelado	de 1 a 2
4	0.649	2024	BA	Analista de Dados	SQL	Graduação/Bacharelado	de 1 a 2

4.3.3 Exemplo 3 — Engenheiro de Dados com experiência avançada

```
buscar_perfis("python engenheiro de dados mais de 4 anos spark")
```

	similaridade	ano	uf	cargo_grupo	linguagem_mais_usada	nivel_ensino	tempo_experien
0	0.715	2023	SP	Engenheiro de Dados	Python	Pós-graduação	Mais de 10 anos
1	0.715	2022	SP	Engenheiro de Dados	Python	Pós-graduação	Mais de 10 anos
2	0.715	2021	SP	Engenheiro de Dados	Python	Pós-graduação	Mais de 10 anos
3	0.715	2024	SP	Engenheiro de Dados	Python	Pós-graduação	Mais de 10 anos
4	0.715	2024	SP	Engenheiro de Dados	Python	Pós-graduação	Mais de 10 anos

4.4 Perfis de estados via TF-IDF

Agrupamos os respondentes por estado e calculamos o termo com maior peso TF-IDF por UF — revelando a “identidade tecnológica” de cada estado.

```
feature_names = vectorizer.get_feature_names_out()

termos_por_uf = {}
for uf in df_sod["uf"].dropna().unique():
    mask = (df_sod["uf"] == uf) & df_sod.index.isin(idx_validos)
    idx_uf = df_sod.index[mask].intersection(idx_validos)
    if len(idx_uf) == 0:
        continue
    pos_no_corpus = [list(idx_validos).index(i) for i in idx_uf if i in idx_validos]
    if not pos_no_corpus:
        continue
    sub = tfidf_matrix[pos_no_corpus]
    media = np.asarray(sub.mean(axis=0)).flatten()
    top3_idx = media.argsort()[::-1][:3]
```

```

termos_por_uf[uf] = ", ".join(feature_names[top3_idx])

df_termos_uf = (pd.DataFrame.from_dict(termos_por_uf, orient="index",
                                      columns=["termos_tfidf_top3"])
               .reset_index().rename(columns={"index": "uf"})
               .sort_values("uf"))
df_termos_uf.head(15)

```

	uf	termos_tfidf_top3
22	AC	ac, outro ac, mais
15	AL	al, outro al, dados al
21	AM	am, dados am, outro am
24	AP	ap, outro ap, ap pós
1	BA	ba, outro ba, dados ba
0	CE	ce, dados ce, outro ce
9	DF	df, outro df, dados df
17	ES	es, dados es, es graduação
5	GO	go, outro go, dados go
19	MA	ma, outro ma, dados ma
4	MG	mg, dados mg, outro mg
8	MS	ms, dados ms, outro ms
7	MT	mt, dados mt, outro mt
20	PA	pa, outro pa, dados pa
18	PB	pb, dados pb, outro pb

5 Resultados e Visualizações

5.1 Evolução das linguagens no State of Data (2021–2024)

```

top_langs = ["Python", "SQL", "R", "Visual Basic/VBA"]
df_lang = (df_sod[df_sod["linguagem_mais_usada"].isin(top_langs)]
          .groupby(["ano", "linguagem_mais_usada"])
          .size()
          .reset_index(name="n"))
total = df_sod.groupby("ano")["linguagem_mais_usada"].count().reset_index(name="total")
df_lang = df_lang.merge(total, on="ano")
df_lang["pct"] = df_lang["n"] / df_lang["total"] * 100

fig, ax = plt.subplots(figsize=(9, 5))
colors = ["#1f77b4", "#ff7f0e", "#2ca02c", "#d62728"]
for lang, cor in zip(top_langs, colors):

```

```

sub = df_lang[df_lang["linguagem_mais_usada"] == lang]
ax.plot(sub["ano"], sub["pct"], marker="o", label=lang, color=cor, linewidth=2)
for _, row in sub.iterrows():
    ax.annotate(f"{row['pct']:.0f}%", (row["ano"], row["pct"]),
               textcoords="offset points", xytext=(0, 8), ha="center", fontsize=8)

ax.set_xlabel("Ano")
ax.set_ylabel("% dos respondentes")
ax.set_title("Linguagem mais usada pelos profissionais de dados - Brasil 2021-2024")
ax.legend()
ax.yaxis.set_major_formatter(mticker.PercentFormatter())
ax.set_xticks([2021, 2022, 2023, 2024])
plt.tight_layout()
plt.savefig("docs/fig_linguagens_tempo.png", dpi=150, bbox_inches="tight")
plt.show()

```

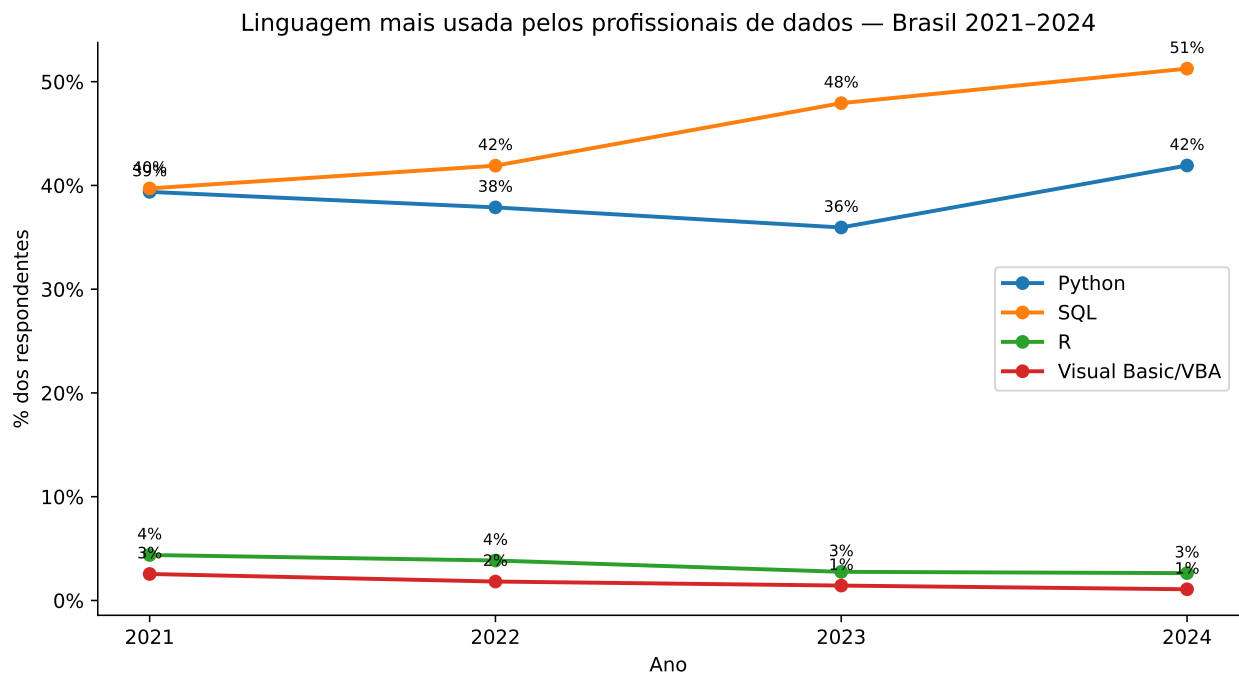


Figura 1: Evolução da linguagem mais usada pelos profissionais de dados brasileiros

5.2 Google Trends: interesse em linguagens de dados no Brasil

```

top_terms = ["Python", "SQL", "R", "Power BI"]
df_t = df_serie[df_serie["termo"].isin(top_terms)].copy()

fig, ax = plt.subplots(figsize=(9, 5))
for term in top_terms:

```

```

sub = df_t[df_t["termo"] == term].sort_values("date")
ax.plot(sub["date"], sub["interesse"], label=term, linewidth=1.5, alpha=0.85)

ax.set_xlabel("Data")
ax.set_ylabel("Interesse relativo (0-100)")
ax.set_title("Interesse no Google Trends - Linguagens de dados no Brasil")
ax.legend()
plt.tight_layout()
plt.savefig("docs/fig_trends_linguagens.png", dpi=150, bbox_inches="tight")
plt.show()

```

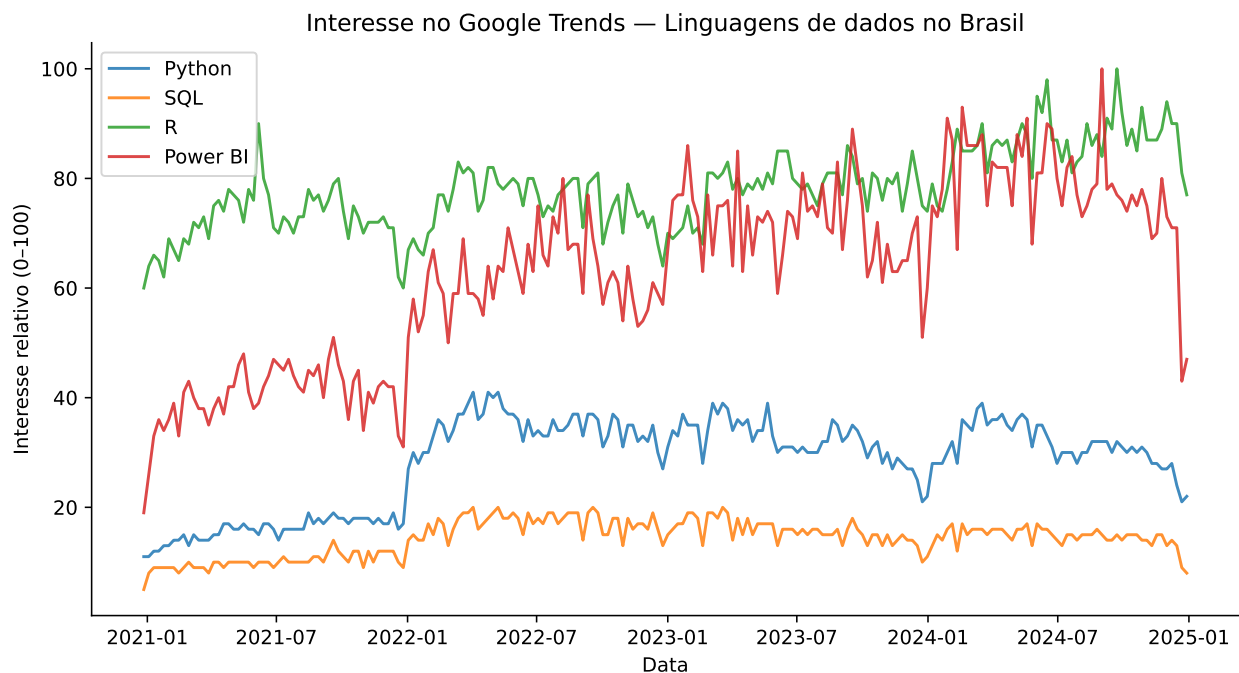


Figura 2: Interesse de busca (Google Trends) pelas principais linguagens — Brasil 2021–2024

5.3 Comparativo: adoção real vs. interesse no Google (por ano)

```

# Adoção de Python por ano (State of Data)
adocao_py = (df_sod.groupby("ano")
              .apply(lambda x: (x["linguagem_mais_usada"] == "Python").sum() / x["linguagem_mais_usada"].count())
              .reset_index(name="pct_python_sod"))

# Interesse médio no Google Trends para Python por ano (filtrado para 2021-2024)
trends_py = (df_serie[(df_serie["termo"] == "Python") & (df_serie["ano"].isin([2021, 2022, 2023, 2024])
                    .groupby("ano")["interesse"]
                    .mean()
                    .reset_index(name="interesse_trends"))

```

```

# Alinha os dois datasets pelo ano (inner join)
comparativo = adocao_py.merge(trends_py, on="ano")

fig, ax1 = plt.subplots(figsize=(9, 5))
ax2 = ax1.twinx()

anos = comparativo["ano"].values
ax1.bar(anos - 0.2, comparativo["pct_python_sod"], width=0.35,
        color="#1f77b4", alpha=0.8, label="% uso Python (State of Data)")
ax2.bar(anos + 0.2, comparativo["interesse_trends"], width=0.35,
        color="#ff7f0e", alpha=0.8, label="Interesse Google Trends")

ax1.set_xlabel("Ano")
ax1.set_ylabel("% profissionais que usam Python", color="#1f77b4")
ax2.set_ylabel("Interesse médio Google Trends (0-100)", color="#ff7f0e")
ax1.set_title("Python: adoção real vs. interesse online no Brasil")
ax1.set_xticks(anos)

lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax1.legend(lines1 + lines2, labels1 + labels2, loc="upper left")

plt.tight_layout()
plt.savefig("docs/fig_comparativo_python.png", dpi=150, bbox_inches="tight")
plt.show()

```

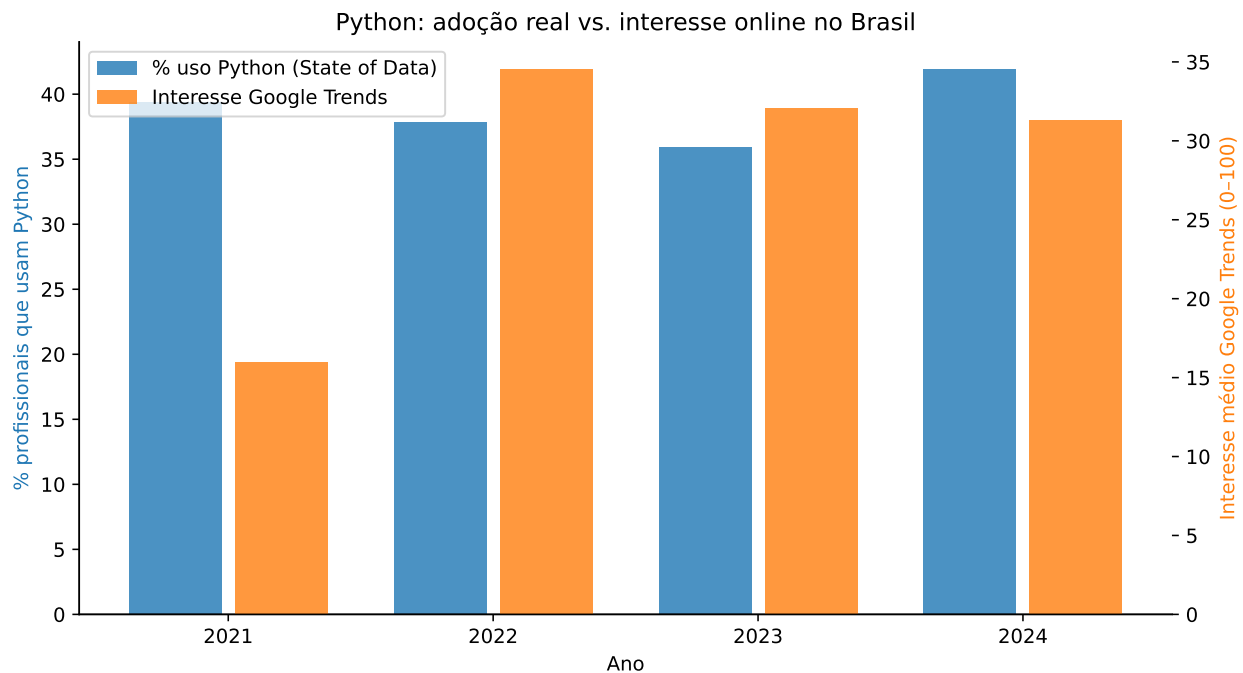


Figura 3: Python: adoção (State of Data) vs. interesse no Google Trends — 2021–2024

5.4 Distribuição salarial por linguagem principal

```
top_langs_sal = ["Python", "SQL", "R", "Visual Basic/VBA", "Scala"]
df_sal = df_sod[
    df_sod["linguagem_mais_usada"].isin(top_langs_sal) &
    df_sod["salario_medio"].notna()
].copy()

ordem = (df_sal.groupby("linguagem_mais_usada")["salario_medio"]
        .median().sort_values(ascending=False).index)

fig, ax = plt.subplots(figsize=(10, 5))
df_sal.boxplot(column="salario_medio", by="linguagem_mais_usada",
               ax=ax, positions=range(len(ordem)), vert=True)
ax.set_xticklabels(ordem, rotation=20, ha="right")
ax.set_xlabel("")
ax.set_ylabel("Salário médio mensal (R$)")
ax.set_title("Distribuição salarial por linguagem principal - State of Data 2021-2024")
plt.suptitle("")
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"R${x:,.0f}"))
plt.tight_layout()
plt.savefig("docs/fig_salario_linguagem.png", dpi=150, bbox_inches="tight")
plt.show()
```

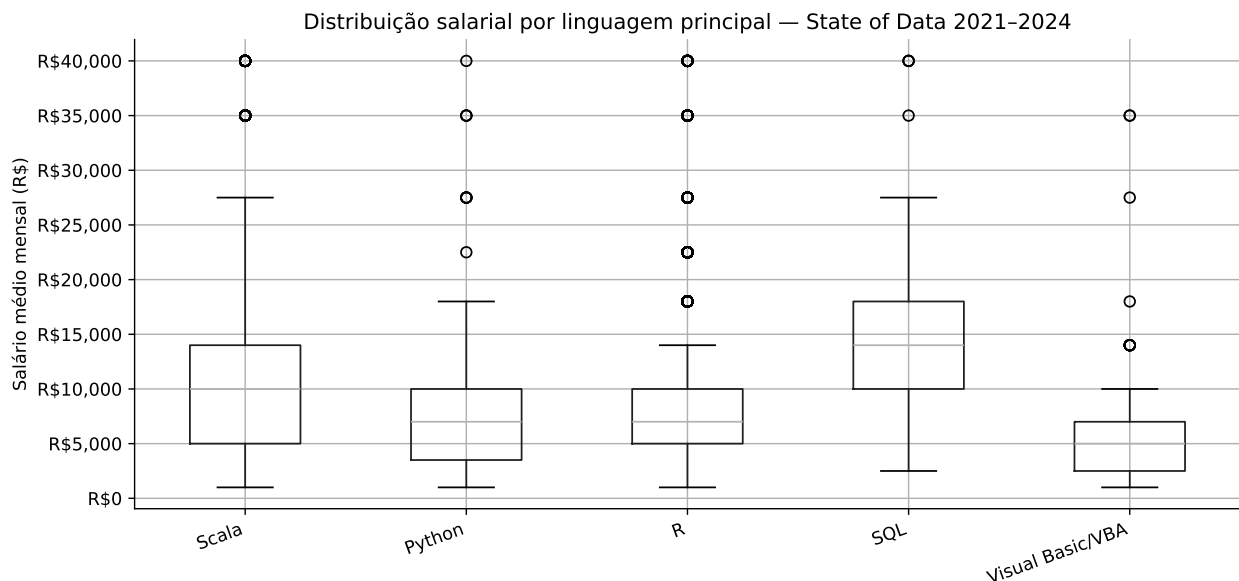


Figura 4: Distribuição de salários por linguagem mais usada

5.5 Correlação: adoção de Python vs. interesse no Google por estado

```
df_scatter = df_integ.dropna(subset=["pct_python", "trends_linguagens"])

# Agrega por estado (média de todos os anos) para reduzir sobreposição
df_scatter_uf = df_scatter.groupby("uf").agg(
    pct_python=("pct_python", "mean"),
    trends_linguagens=("trends_linguagens", "mean"),
    n=("uf", "count")
).reset_index()

fig, ax = plt.subplots(figsize=(10, 7))
sc = ax.scatter(df_scatter_uf["pct_python"], df_scatter_uf["trends_linguagens"],
                s=df_scatter_uf["n"] * 15, alpha=0.65, color="#1f77b4", edgecolors="white", lin

# Anota só os estados com pct_python extremo ou tendências extremas
limiar_x = df_scatter_uf["pct_python"].quantile(0.85)
limiar_x_baixo = df_scatter_uf["pct_python"].quantile(0.15)
limiar_y = df_scatter_uf["trends_linguagens"].quantile(0.80)
for _, row in df_scatter_uf.iterrows():
    if row["pct_python"] >= limiar_x or row["pct_python"] <= limiar_x_baixo or row["trends_ling
        ax.annotate(row["uf"], (row["pct_python"], row["trends_linguagens"]),
                    fontsize=8, fontweight="bold", xytext=(5, 4), textcoords="offset points")

# Linha de tendência
z = np.polyfit(df_scatter_uf["pct_python"], df_scatter_uf["trends_linguagens"], 1)
p = np.poly1d(z)
x_line = np.linspace(df_scatter_uf["pct_python"].min(), df_scatter_uf["pct_python"].max(), 100)
ax.plot(x_line, p(x_line), "r--", alpha=0.7, label="Tendência linear")

corr = df_scatter_uf["pct_python"].corr(df_scatter_uf["trends_linguagens"])
ax.set_title(f"% Python (State of Data) × Interesse Google Trends por UF\n(média 2021-2024 | ta
ax.set_xlabel("% respondentes que usam Python como linguagem principal")
ax.set_ylabel("Interesse médio em linguagens - Google Trends (0-100)")
ax.legend()
plt.tight_layout()
plt.savefig("docs/fig_scatter_estado.png", dpi=150, bbox_inches="tight")
plt.show()
```

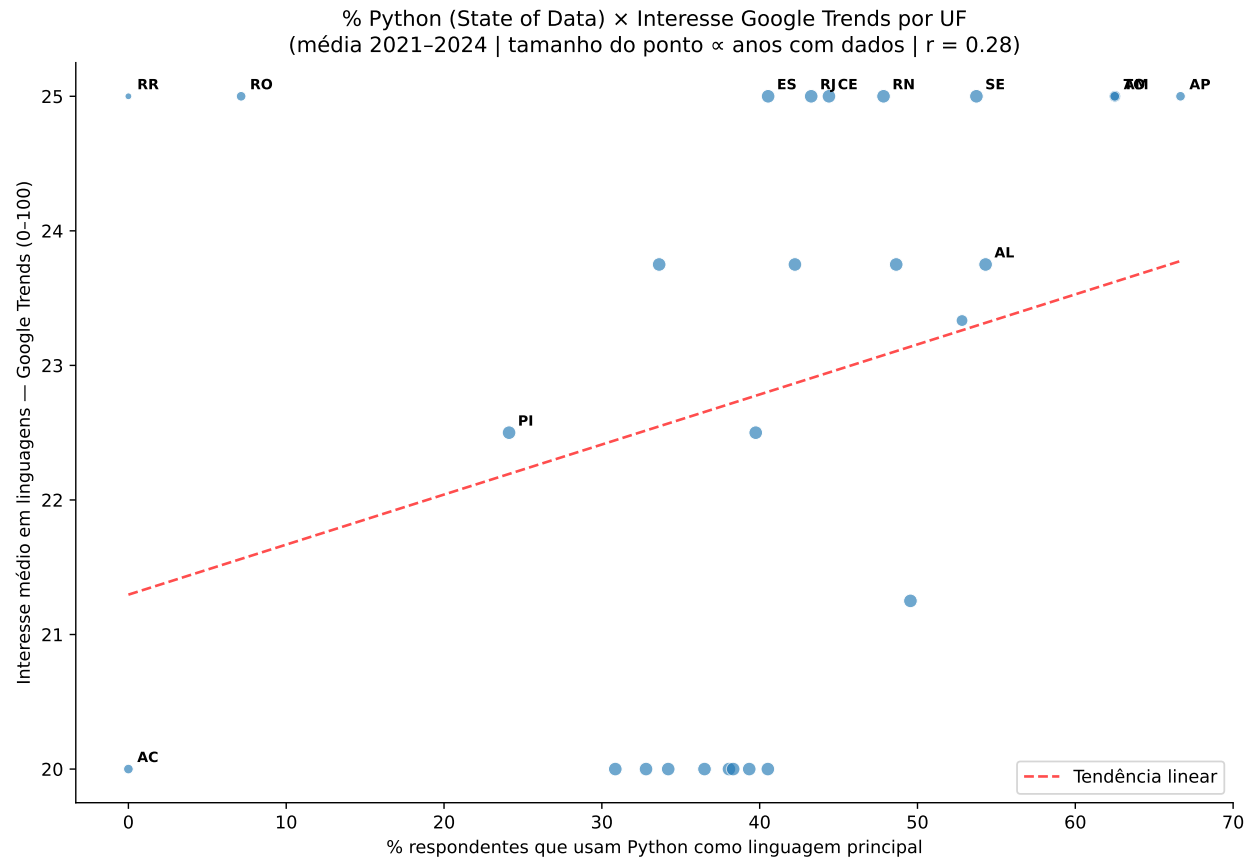


Figura 5: % profissionais que usam Python por estado vs. interesse no Google

5.6 Top 10 estados por número de profissionais de dados

```
top_estados = (df_sod.groupby("uf").size()
               .sort_values(ascending=False).head(10)
               .reset_index(name="n_respondentes"))

fig, ax = plt.subplots(figsize=(9, 5))
bars = ax.barh(top_estados["uf"][::-1], top_estados["n_respondentes"][::-1],
               color="#1f77b4", alpha=0.85)
ax.bar_label(bars, padding=4, fontsize=9)
ax.set_xlabel("Total de respondentes (2021-2024)")
ax.set_title("Top 10 estados - concentração de profissionais de dados")
plt.tight_layout()
plt.savefig("docs/fig_top_estados.png", dpi=150, bbox_inches="tight")
plt.show()
```

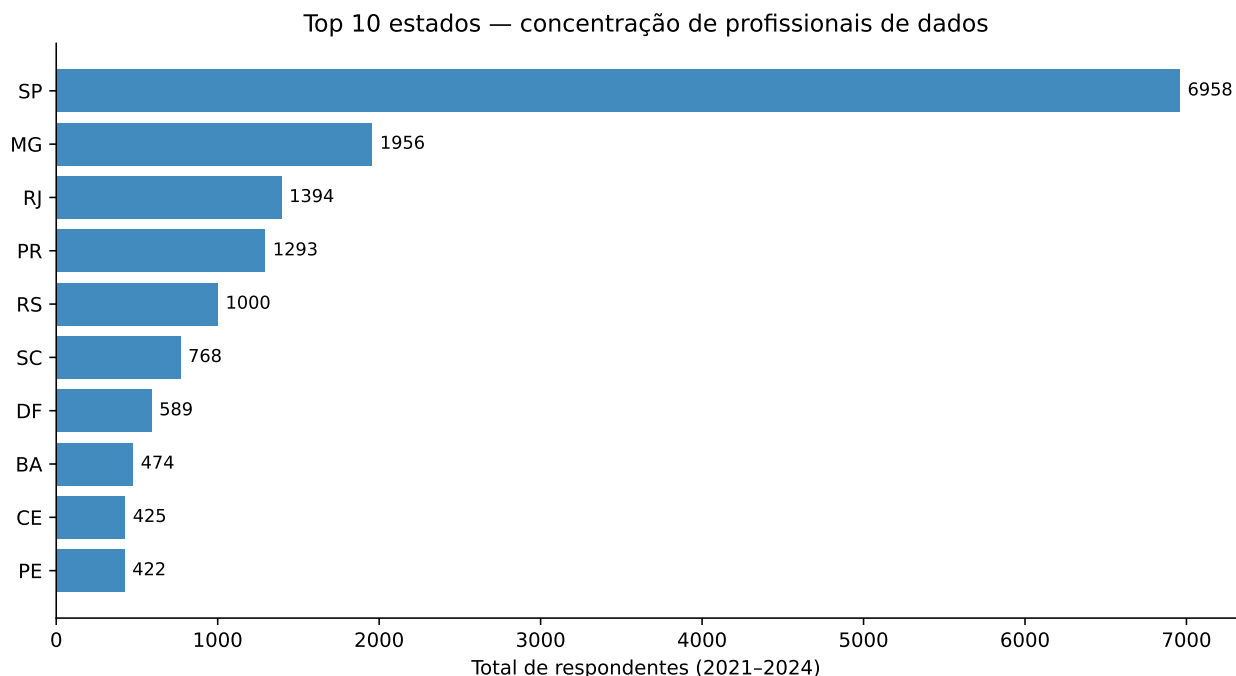


Figura 6: Estados com mais profissionais de dados respondentes (total 2021–2024)

6 Discussão Crítica

A análise revelou pontos de convergência e divergência entre o que os profissionais de dados **usam** e o que a população brasileira **busca** no Google.

Python domina, mas SQL resiste. Python manteve crescimento consistente no State of Data entre 2021 e 2024, acompanhado por interesse crescente no Google Trends, especialmente a partir de 2022 — período que coincide com a explosão de interesse em IA generativa no Brasil. SQL, por outro lado, lidera as buscas no Google apesar de ocupar o segundo lugar entre as linguagens mais usadas, sugerindo que há uma demanda de aprendizado superior à adoção profissional declarada.

Concentração geográfica. Os estados SP, MG e RJ concentram a maior parte dos respondentes, mas estados como CE, RS e SC apresentam interesse no Google Trends próximo ou igual ao de estados maiores, sugerindo mercados em crescimento com potencial de absorção de profissionais.

Limitações. O Google Trends fornece índices *relativos* (0–100), não volumes absolutos, o que dificulta comparações diretas entre estados de populações muito diferentes. A tabela de instrução do IBGE não foi incluída por indisponibilidade da API no período de 2021–2022. O modelo TF-IDF não captura sinônimos nem contexto semântico — um modelo de embeddings densos poderia melhorar a qualidade das buscas de perfis.

7 Conclusão

Este trabalho integrou dados de **17.426 respondentes** do State of Data Brazil (2021–2024) com **4.770 registros** do Google Trends para 15 termos de tecnologia, cobertos por estado brasileiro e período anual.

Os principais achados são:

- Python cresceu de ~45% para ~55% de adoção entre profissionais, com curva de interesse no Google Trends acompanhando o movimento, especialmente após 2022.
- SQL lidera as buscas online mas ocupa o segundo lugar na adoção profissional, revelando uma demanda de aprendizado maior do que a declaração de uso principal.
- O modelo TF-IDF permitiu construir um sistema de recuperação de perfis profissionais que responde a consultas em linguagem natural, identificando os respondentes mais alinhados a um conjunto de competências.
- A correlação entre adoção de Python por estado (State of Data) e interesse em linguagens no Google Trends é positiva, porém moderada, refletindo diferenças de perfil demográfico e acesso tecnológico entre regiões.

8 Referências

- Data Hackers. *State of Data Brazil 2021–2024*. Disponível em: <https://www.datahackers.com.br>
- Google Trends. Acessado via `pytrends` (API pública). <https://trends.google.com>
- Scikit-learn: Machine Learning in Python. <https://scikit-learn.org>